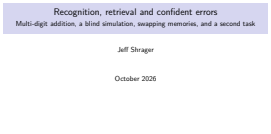
Recognition, retrieval and confident errors

Multi-digit addition, a blind simulation, swapping memories, and a second task

Jeff Shrager

October 2026

Recognition, retrieval and confident errors



This talk follows one model from its postulates to a theorem that two different tasks inherit. It's a model of multi-digit addition that combines two classic theories, EPAM and the distributions-of-associations model. I'll state its central prediction, confident errors, show a worked case, describe how we checked the formalization with a blind simulation, swap EPAM's memory for ACT-R's, extract the general composition, apply it to naming people, and derive the result from laws. Everything I quote is a theorem or an exact computation checked by Lean.

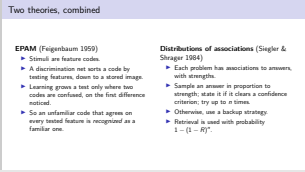
Two theories, combined

EPAM (Feigenbaum 1959)

- Stimuli are feature codes.
- A discrimination net sorts a code by testing features, down to a stored image.
- Learning grows a test only where two codes are confused, on the first difference noticed.
- So an unfamiliar code that agrees on every tested feature is *recognized as* a familiar one.

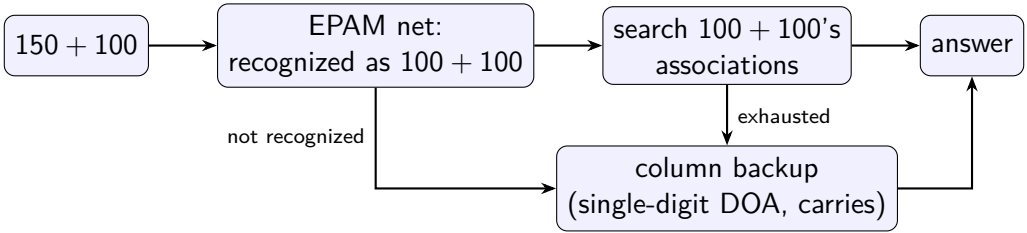
Distributions of associations (Siegler & Shrager 1984)

- Each problem has associations to answers, with strengths.
- Sample an answer in proportion to strength; state it if it clears a confidence criterion; try up to n times.
- Otherwise, use a backup strategy.
- Retrieval is used with probability $1 - (1 - R)^n$.



Here are two classic theories. EPAM, Feigenbaum's model of verbal learning: stimuli are feature codes, sorted through a discrimination net of feature tests, and the net grows a new test only where two codes get confused. So an unfamiliar code that matches a familiar one on every tested feature gets recognized as that familiar one. And the distributions-of-associations model of strategy choice, which Bob Siegler and I proposed: each problem has associations to answers; you sample an answer in proportion to its strength, state it if it clears a confidence criterion, try a few times, and otherwise fall back on a backup strategy such as counting.

The multi-digit model



- Every postulate is a parameter: the encoding, the net, the strengths, the criterion, the search length, the backup.
- 18 predictions, each proved for every choice of them.

The multi-digit model



- Every postulate is a parameter: the encoding, the net, the strengths, the criterion, the search length, the backup.
- 18 predictions, each proved for every choice of them.

Combine them and you get a model of three-digit addition. A problem like 150 plus 100 is first recognized by an EPAM net, perhaps as the familiar 100 plus 100. Then retrieval searches the recognized problem's associations. If nothing is recognized, or retrieval comes up empty, the child works it out column by column, where each column is itself a single-digit trial of retrieval or backup, with carries. Every postulate here is a parameter: the encoding, the net, the strengths, the criterion, the search length, the backup. The model yields eighteen predictions, each proved for every choice of those parameters.

Confident errors

Let a familiar problem q have *sharp* associations (only its correct sum clears the criterion), and let x, with a different sum, be recognized as q. Then:

1. on q, whole-problem retrieval never errs;
2. on x, every whole-problem answer is wrong;
3. x uses the backup exactly as rarely as q does: $(1 - R_q)^n$.

In DOA alone, errors and backup use rise together. Confusion breaks the tie: a retrieved, wrong answer with no extra backup use.

Confident errors

Let a familiar problem q have *sharp* associations (only its correct sum clears the criterion), and let x, with a different sum, be recognized as q. Then:

1. on q, whole-problem retrieval never errs;
2. on x, every whole-problem answer is wrong;
3. x uses the backup exactly as rarely as q does: $(1 - R_q)^a$.

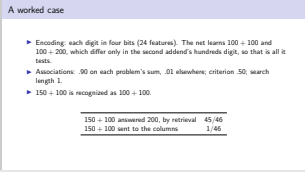
In DOA alone, errors and backup use rise together. Confusion breaks the tie: a retrieved, wrong answer with no extra backup use.

Here is the prediction neither theory makes alone. Suppose a familiar problem q has sharp associations, meaning only its correct sum clears the criterion, and a stranger x with a different sum is recognized as q. Then q is never answered wrongly by retrieval; every retrieved answer to x is wrong; and x falls back on the columns exactly as rarely as q does. In the associations model alone, errors and backup use rise and fall together. Confusion breaks that tie: you get a retrieved, wrong answer with no extra backup use. Notice that I don't say fast: the model has no timing, so that would need more.

A worked case

- Encoding: each digit in four bits (24 features). The net learns $100 + 100$ and $100 + 200$, which differ only in the second addend's hundreds digit, so that is all it tests.
- Associations: .90 on each problem's sum, .01 elsewhere; criterion .50; search length 1.
- $150 + 100$ is recognized as $100 + 100$.

$150 + 100$ answered 200, by retrieval	$45/46$
$150 + 100$ sent to the columns	$1/46$



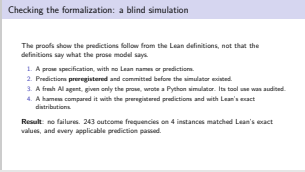
To see it concretely: encode each digit in four bits. A net that has learned 100 plus 100 and 100 plus 200 only needs to test the second addend's hundreds digit to tell them apart. So 150 plus 100 slips through as 100 plus 100. With these strengths and a single search, it's answered 200 by retrieval with probability 45 over 46, and sent to the columns with probability 1 over 46. With the column problem solver doing the columns from single-digit facts, it's then right about two thirds of the time, so correct overall only about one and a half percent of the time. These numbers are just an illustration, computed exactly; the structure, confident errors, holds for every choice of parameters.

Checking the formalization: a blind simulation

The proofs show the predictions follow from the Lean definitions, not that the definitions say what the prose model says.

1. A prose specification, with no Lean names or predictions.
2. Predictions **preregistered** and committed before the simulator existed.
3. A fresh AI agent, given only the prose, wrote a Python simulator. Its tool use was audited.
4. A harness compared it with the preregistered predictions and with Lean's exact distributions.

Result: no failures. 243 outcome frequencies on 4 instances matched Lean's exact values, and every applicable prediction passed.



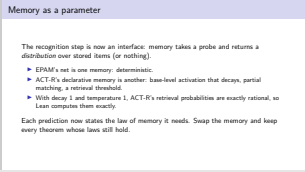
The proofs show that the predictions follow from the Lean definitions, not that the definitions say what the prose model says. So we checked that link with a blind simulation. We wrote a prose specification with no Lean in it, and preregistered the predictions before any simulator existed. A fresh AI agent, given only the prose, wrote a Python simulator, and a harness compared it with the predictions and with Lean's exact distributions. There were no failures: 243 outcome frequencies matched Lean's exact values. It also found five vacuous predictions, a gap in the formalization, and a prediction that depends on a choice the model leaves open. There's a separate talk on this.

Memory as a parameter

The recognition step is now an interface: memory takes a probe and returns a *distribution* over stored items (or nothing).

- EPAM's net is one memory: deterministic.
- ACT-R's declarative memory is another: base-level activation that decays, partial matching, a retrieval threshold.
- With decay 1 and temperature 1, ACT-R's retrieval probabilities are exactly rational, so Lean computes them exactly.

Each prediction now states the law of memory it needs. Swap the memory and keep every theorem whose laws still hold.



Next we made memory a parameter. The recognition step is now an interface: memory takes a probe and returns a probability distribution over stored items, or nothing. EPAM's net is one memory, and it's deterministic. ACT-R's declarative memory is another, with base-level activation that decays, partial matching and a retrieval threshold. With decay one and temperature one, ACT-R's retrieval probabilities are exactly rational, so Lean can compute them exactly. Each prediction now states the law of memory it needs, so when we swap memories we keep every theorem whose laws still hold.

The same problems, two memories

	EPAM	ACT-R
$150 + 100$ answered 200 (the neighbour's sum)	45/46	225/529
$150 + 100$ answered 300 (differs on a tested digit)	0	225/1058
$150 + 100$ sent to the columns	1/46	383/1058
errors on the practiced $100 + 100$	0	225/782
the same, after also studying $100 + 300$	0	3375/5842

ACT-R: confusion is graded, nothing is retrieved for certain, and new practice interferes with old.

The same problems, two memories

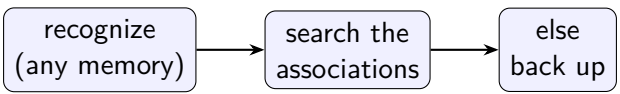
	EPAM	ACT-R
$150 \div 100$ answered 200 (the neighbour's sum)	45/46	225/529
$150 \div 100$ answered 300 (differs on a tested digit)	0	225/1058
$150 \div 100$ sent to the columns	1/46	383/1058
errors on the practiced $100 \div 100$	0	225/782
the same, after also studying $100 \div 300$	0	3375/5842

ACT-R: confusion is graded, nothing is retrieved for certain, and new practice interferes with old.

Here are the same problems run through both memories. With EPAM, 150 plus 100 is answered 200 with probability 45 over 46, and never 300; a practiced problem never errs, and learning something new doesn't change that. With ACT-R, the confusion is graded: 150 plus 100 can be taken for either familiar problem, even the practiced 100 plus 100 sometimes errs, and studying 100 plus 300 more than doubles its errors. EPAM's zeros are theorems. ACT-R's departures are proved on this instance and explained by laws it lacks.

The composition, extracted

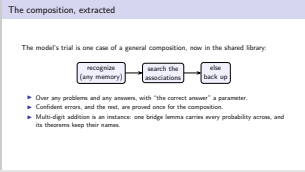
The model's trial is one case of a general composition, now in the shared library:



- Over any problems and any answers, with "the correct answer" a parameter.
- Confident errors, and the rest, are proved once for the composition.
- Multi-digit addition is an instance: one bridge lemma carries every probability across, and its theorems keep their names.

Recognition, retrieval and confident errors

The composition, extracted



The model's trial turns out to be one case of a general composition: recognize with any memory, search the recognized item's associations, else back up. We moved that into the shared library, over any problems and any answers, with the correct answer as a parameter, and proved confident errors there, once. Multi-digit addition is now just an instance: one bridge lemma carries every probability across, and all its theorems keep their names. Its definitions were left untouched, so the numbers exported for the simulation and the paper are byte-identical.

A second task: naming people

Faces are codes; familiar Ann (000) and Bob (110); stranger Carl (100), one feature from each. The backup is "don't know".

	EPAM	ACT-R
Carl named Bob	45/46	225/391
Carl named Ann	0	225/782
Carl: "don't know"	1/46	107/782
Bob: "don't know"	1/46	229/2254
Bob named Ann	0	225/2254

With EPAM, *confident misnaming* is inherited from the composition's theorem, for every search length. No arithmetic anywhere.

A second task: naming people

Faces are codes; familiar Ann (000) and Bob (110); stranger Carl (100), one feature from each. The backup is "don't know".

	EPAM	ACT-R
Carl named Bob	45/46	225/391
Carl named Ann	0	225/782
Carl: "don't know"	1/46	107/782
Bob: "don't know"	1/46	229/2254
Bob named Ann	0	225/2254

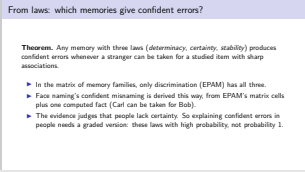
With EPAM, *confident misnaming* is inherited from the composition's theorem, for every search length. No arithmetic anywhere.

To show it isn't about arithmetic, here is a second task through the same composition: naming people. Faces are codes; Ann and Bob are familiar; Carl is a stranger who resembles both. With EPAM's memory, the confident-errors theorem is simply inherited: Carl is confidently misnamed Bob, and he says don't know exactly as rarely as Bob himself does. With ACT-R's memory, the misnaming is graded: Carl can be taken for either familiar person, and he says don't know more often than Bob.

From laws: which memories give confident errors?

Theorem. Any memory with three laws (*determinacy*, *certainty*, *stability*) produces confident errors whenever a stranger can be taken for a studied item with sharp associations.

- In the matrix of memory families, only discrimination (EPAM) has all three.
- Face naming's confident misnaming is derived this way, from EPAM's matrix cells plus one computed fact (Carl can be taken for Bob).
- The evidence judges that people lack certainty. So explaining confident errors in people needs a graded version: these laws with high probability, not probability 1.



Here's where the two halves meet. Confident errors can be derived from three laws: any memory with determinacy, certainty and stability produces them, whenever a stranger can be taken for a studied item. So the matrix answers the question of which memory families produce confident errors: only discrimination. And since the evidence suggests people lack certainty, explaining confident errors in people will need a graded version of this theorem.

Limits, and next

- **No timing.** "Retrieved, with no extra backup use" is what is proved; "fast" needs a timing model.
- **One trial at a time.** Learning across trials (feedback, discrimination, strengthening) makes memory states random; the predicted sequence (confident error, then column work, then fluent retrieval) is the next target.
- **Formal fidelity.** ACT-R here is one restricted formalization; checking it against PyACTUp, the community implementation, is planned.
- **A graded confident-errors theorem**, for memories without certainty.

Limits, and next

- **No timing.** "Retrieved, with no extra backup use" is what is proved; "fast" needs a timing model.
- **One trial at a time.** Learning across trials (feedback, discrimination, strengthening) makes memory states random; the predicted sequence (confident error, then column work, then fluent retrieval) is the next target.
- **Formal fidelity.** ACT-R here is one restricted formalization; checking it against PyACTUp, the community implementation, is planned.
- **A graded confident-errors theorem**, for memories without certainty.

Some limits. The model has no timing, so what is proved is a retrieved answer with no extra backup use, not a fast one. It treats one trial at a time: learning across trials, where feedback discriminates and strengthens, makes memory states random, and the predicted sequence of a confident error, then column work, then fluent retrieval is the next target. Our ACT-R is one restricted formalization, which we plan to check against PyACTUp. And we need a graded version of the confident-errors theorem, for memories that are never certain.